

Assignment 22.3

Name: B. Sarayu

Hall Ticket No: 2303A51842

Task 1 – Bias Detection in AI Suggestions

Prompt Used:

Generate a resume shortlisting algorithm to select best candidate.

Generate a resume shortlisting algorithm ensuring diversity.

Explanation:

The outputs varied. The first prioritized high experience and top universities.

The second included diversity factors like varied backgrounds.

Ethical Insight:

AI may introduce bias depending on prompt wording.

Code:

```
# Resume shortlisting simulation
```

```
candidates = [  
    {"name": "A", "college": "IIT", "experience": 5},  
    {"name": "B", "college": "Local College", "experience": 5},  
    {"name": "C", "college": "IIT", "experience": 2},  
    {"name": "D", "college": "Local College", "experience": 6}  
]
```

```
print("=== Shortlisting based on 'Best Candidate' ===")
```

```
for c in candidates:
```

```
if c["college"] == "IIT" and c["experience"] >= 3:
```

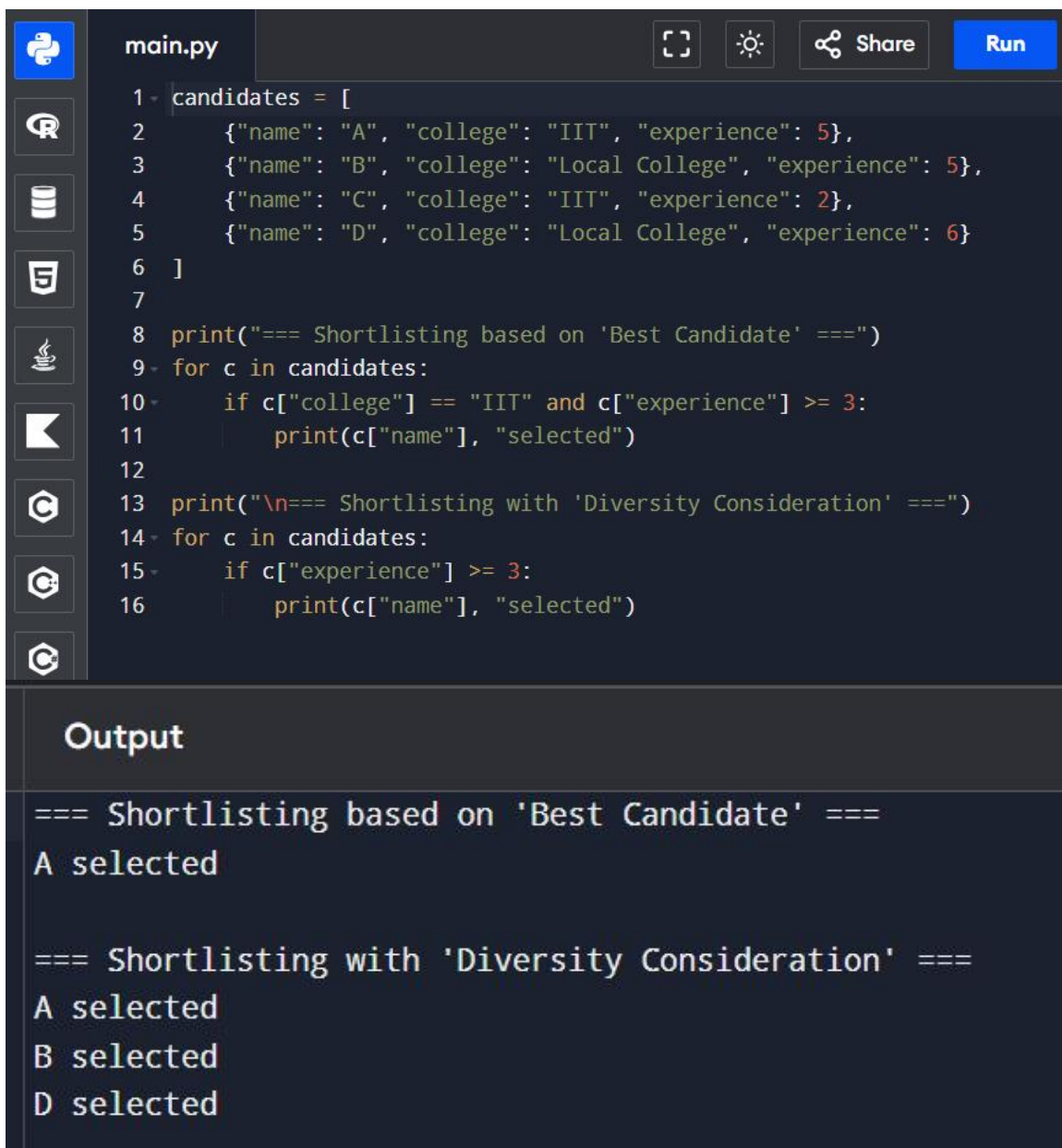
```
    print(c["name"], "selected")
```

```
print("\n=== Shortlisting with 'Diversity Consideration' ===")
```

```
for c in candidates:
```

```
    if c["experience"] >= 3:
```

```
        print(c["name"], "selecte
```

A screenshot of a Python IDE interface. The top bar shows a file named 'main.py' with icons for full screen, settings, and a 'Share' button, followed by a blue 'Run' button. The left sidebar contains icons for various tools like a debugger, database, and terminal. The main editor area displays Python code with line numbers 1 through 16. The code defines a list of candidates and performs two filtering operations. The first operation prints 'A selected' for candidate A. The second operation prints 'A selected', 'B selected', and 'D selected' for candidates A, B, and D respectively. The bottom section, titled 'Output', shows the execution results of the code.

```
1 candidates = [  
2     {"name": "A", "college": "IIT", "experience": 5},  
3     {"name": "B", "college": "Local College", "experience": 5},  
4     {"name": "C", "college": "IIT", "experience": 2},  
5     {"name": "D", "college": "Local College", "experience": 6}  
6 ]  
7  
8 print("=== Shortlisting based on 'Best Candidate' ===")  
9 for c in candidates:  
10     if c["college"] == "IIT" and c["experience"] >= 3:  
11         print(c["name"], "selected")  
12  
13 print("\n=== Shortlisting with 'Diversity Consideration' ===")  
14 for c in candidates:  
15     if c["experience"] >= 3:  
16         print(c["name"], "selected")
```

Output

```
=== Shortlisting based on 'Best Candidate' ===  
A selected  
  
=== Shortlisting with 'Diversity Consideration' ===  
A selected  
B selected  
D selected
```

Task 2 – Privacy and Sensitive Data Handling

Prompt Used:

Write code to process student records including roll number, marks, and email.

Code:

```
student = {'roll': '101', 'marks': 90, 'email': 'abc@gmail.com'}
```

Explanation:

Sensitive data should be encrypted or masked.

Ethical Insight:

Privacy laws like GDPR require protection of personal data.

Code:

```
# Student data handling with masking
```

```
student = {  
    "roll": "101",  
    "marks": 92,  
    "email": "sarayu@gmail.com"  
}
```

```
def mask_email(email):  
    parts = email.split("@")  
    return parts[0][:2] + "***@" + parts[1]
```

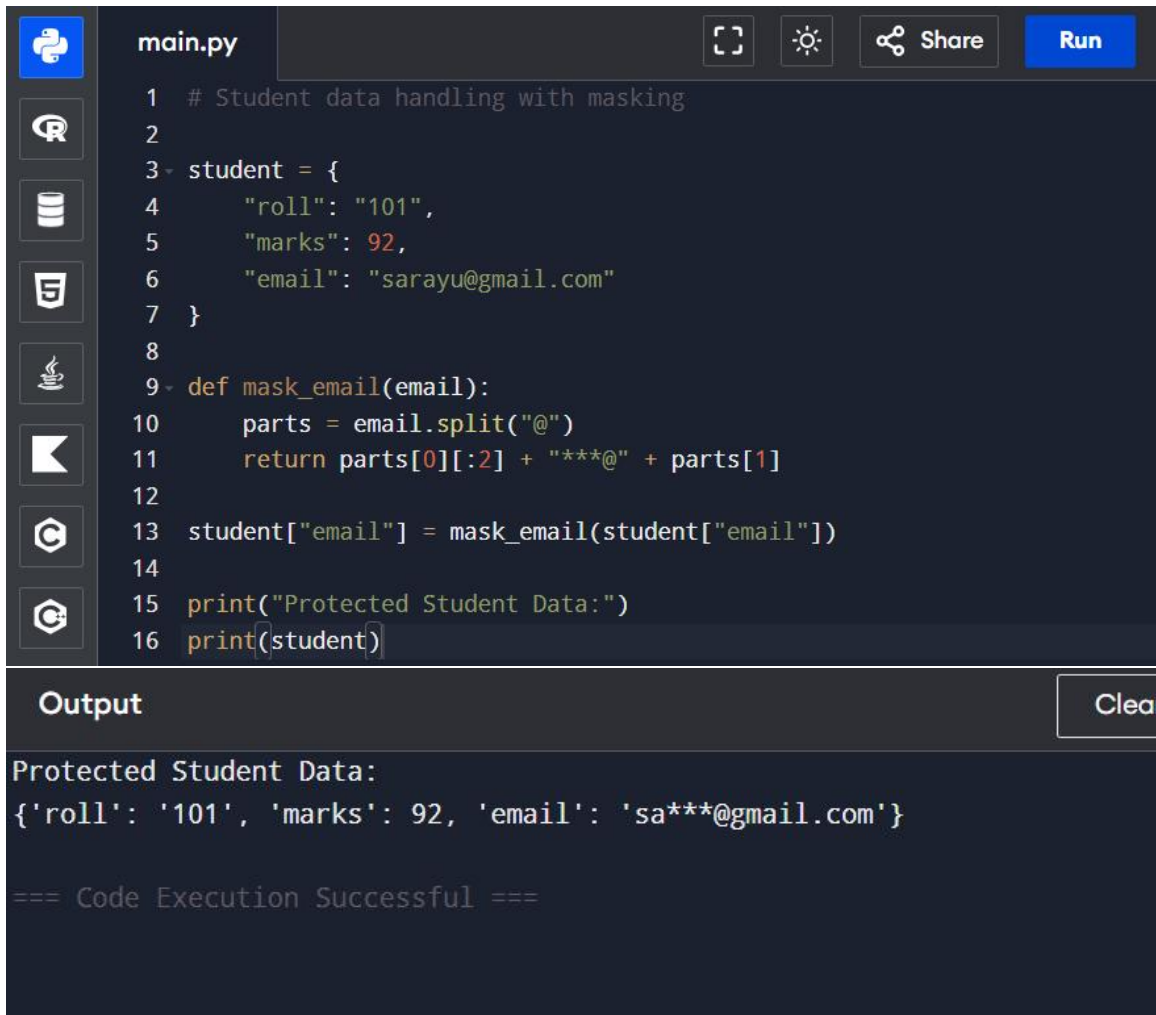
```
student["email"] = mask_email(student["email"])
```

```
print("Protected Student Data:")
```

```
print(student)
```

Output:

{'roll': '101', 'marks': 90, 'email': '***@gmail.com'}



The screenshot shows a Jupyter Notebook interface with a dark theme. The top bar includes a Python logo, a file icon, a settings icon, a share icon, and a 'Run' button. The left sidebar contains icons for various data science tools. The main area displays a Python script named 'main.py' with the following code:

```
1 # Student data handling with masking
2
3 student = {
4     "roll": "101",
5     "marks": 92,
6     "email": "sarayu@gmail.com"
7 }
8
9 def mask_email(email):
10     parts = email.split("@")
11     return parts[0][:2] + "***@" + parts[1]
12
13 student["email"] = mask_email(student["email"])
14
15 print("Protected Student Data:")
16 print(student)
```

Below the code editor is an 'Output' section with a 'Clear' button. The output displays the result of the script execution:

```
Protected Student Data:
{'roll': '101', 'marks': 92, 'email': 'sa***@gmail.com'}
```

At the bottom of the output section, it says '=== Code Execution Successful ==='.

Task 3 – AI-Generated Security Risks

Prompt Used:

Generate payment processing script.

Code:

```
card = input('Enter card number')
```

Explanation:

AI may store sensitive data in plain text.

Risks:

No encryption, no validation.

Ethical Solution:

Use encryption and validation checks.

Code:

```
import re
```

```
card_number = input("Enter card number: ")
```

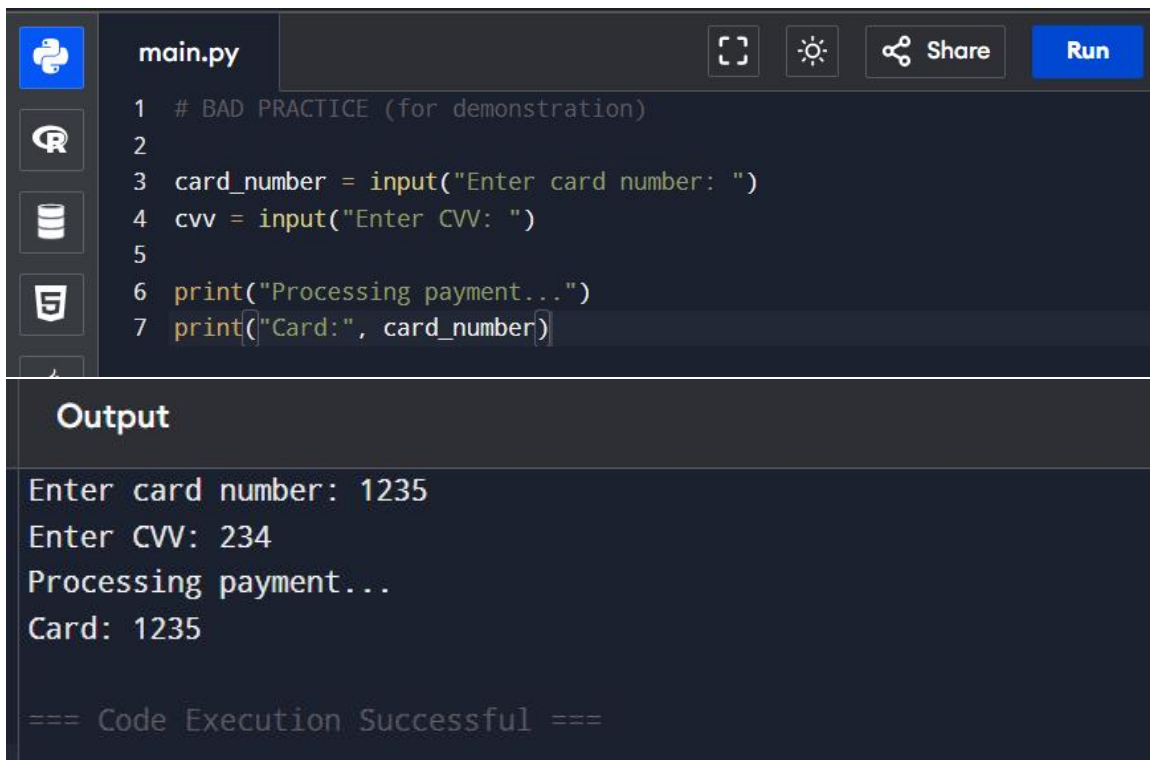
```
if re.fullmatch(r"\d{16}", card_number):
```

```
    print("Valid card format")
```

```
    print("Processing securely...")
```

```
else:
```

```
    print("Invalid card number")
```



The screenshot shows a Python code editor interface. On the left, there is a sidebar with icons for Python, Jupyter Notebook, Database, and a file explorer. The main area displays a file named `main.py` with the following code:

```
1 # BAD PRACTICE (for demonstration)
2
3 card_number = input("Enter card number: ")
4 cvv = input("Enter CVV: ")
5
6 print("Processing payment...")
7 print("Card:", card_number)
```

Below the code editor, there is an "Output" section showing the execution results:

```
Enter card number: 1235
Enter CVV: 234
Processing payment...
Card: 1235

=== Code Execution Successful ===
```

Task 4 – Accountability in Human–AI Collaboration

Scenario:

Traffic system controlled by AI.

Reflection:

Responsibility should be shared between developers, deployers, and regulators.

Code:

```
# Traffic signal decision simulation
```

```
def traffic_signal(cars_waiting):
```

```
    if cars_waiting > 10:
```

```
        return "GREEN"
```

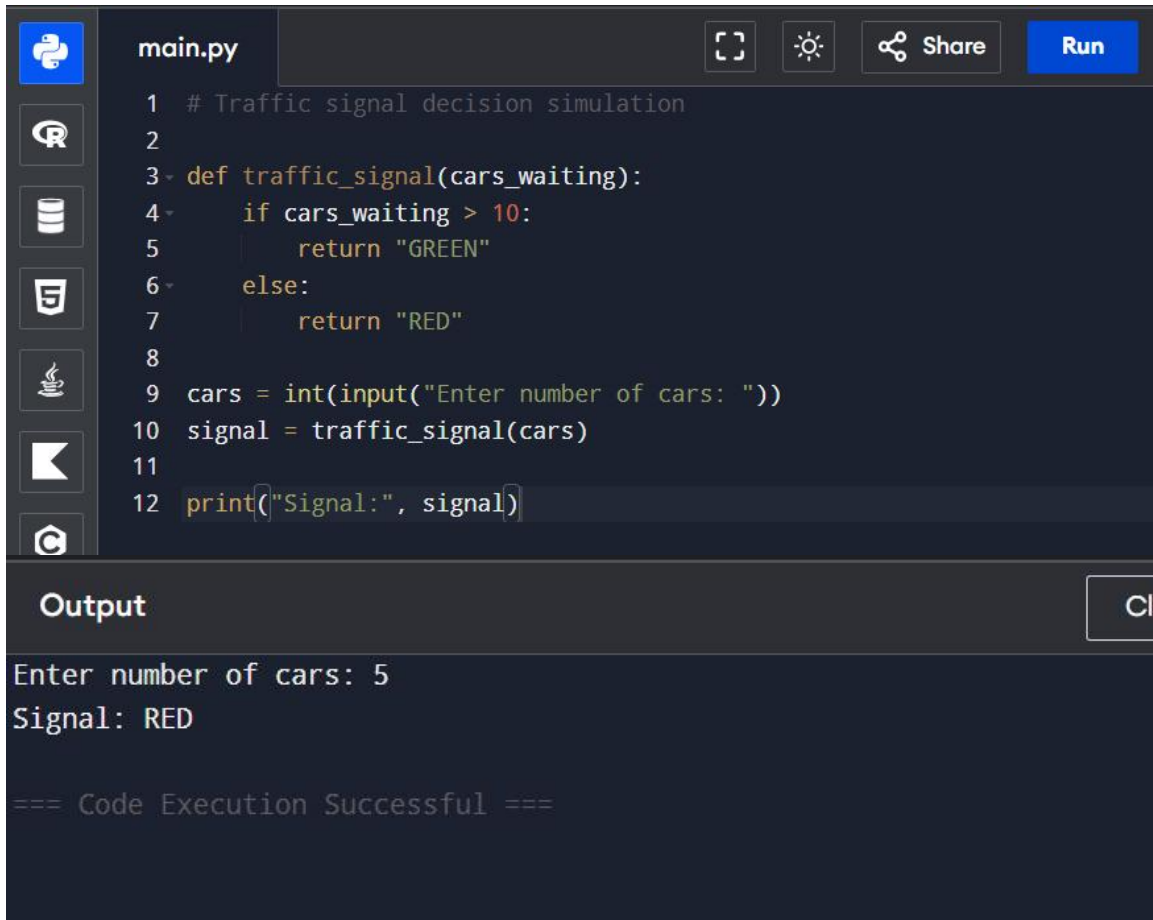
```
    else:
```

```
        return "RED"
```

```
cars = int(input("Enter number of cars: "))
```

```
signal = traffic_signal(cars)
```

```
print("Signal:", signal)
```



The screenshot shows a Python IDE interface. On the left is a sidebar with icons for Python, Jupyter, Docker, VS Code, a coffee cup, a left arrow, and a home icon. The main editor window is titled 'main.py' and contains the following Python code:

```
1 # Traffic signal decision simulation
2
3 def traffic_signal(cars_waiting):
4     if cars_waiting > 10:
5         return "GREEN"
6     else:
7         return "RED"
8
9 cars = int(input("Enter number of cars: "))
10 signal = traffic_signal(cars)
11
12 print("Signal:", signal)
```

Below the code editor is an 'Output' panel. It displays the execution results:

```
Enter number of cars: 5
Signal: RED

=== Code Execution Successful ===
```

At the top right of the editor, there are icons for full screen, settings, and a 'Share' button, followed by a blue 'Run' button.

Task 5 – Limitations of AI in Critical Systems

Scenario:

Aircraft fault detection system.

Explanation:

AI cannot be fully trusted without expert validation.

Conclusion:

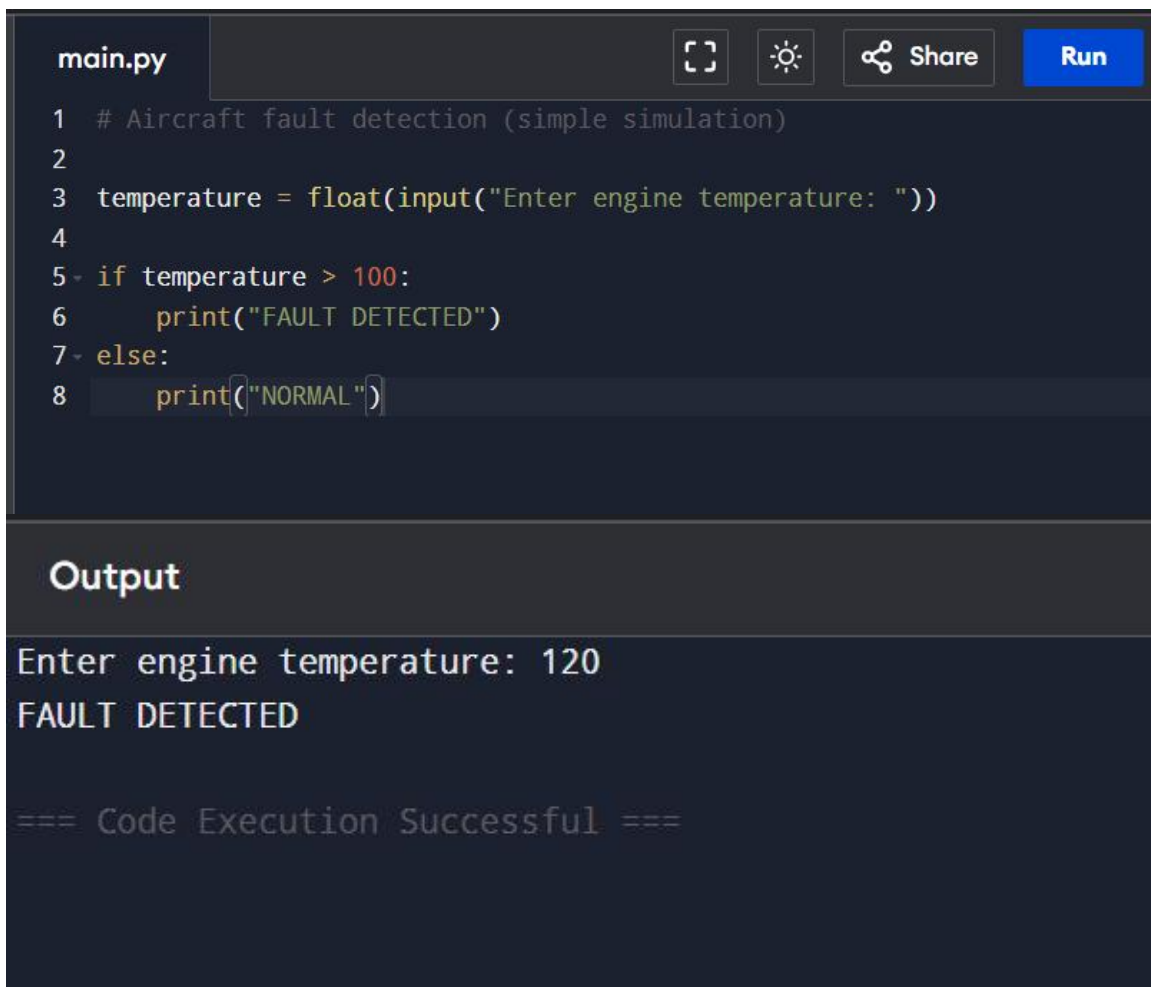
Human oversight is mandatory.

Code:

```
# Aircraft fault detection (simple simulation)
```

```
temperature = float(input("Enter engine temperature: "))
```

```
if temperature > 100:
    print("FAULT DETECTED")
else:
    print("NORMAL")
```



The screenshot displays a code editor window titled 'main.py'. The code is a Python script for a simple aircraft fault detection simulation. It prompts the user to enter the engine temperature. If the temperature is greater than 100, it prints 'FAULT DETECTED'; otherwise, it prints 'NORMAL'. The output section shows the user entered '120', and the program correctly output 'FAULT DETECTED'. The execution was successful.

```
main.py
1 # Aircraft fault detection (simple simulation)
2
3 temperature = float(input("Enter engine temperature: "))
4
5 if temperature > 100:
6     print("FAULT DETECTED")
7 else:
8     print("NORMAL")
```

Output

```
Enter engine temperature: 120
FAULT DETECTED

=== Code Execution Successful ===
```

Task 6 – Ethical Use of AI-Generated Code

Explanation:

Using AI code without understanding is risky.

Best Practices:

Verify code, cite AI usage, avoid plagiarism.

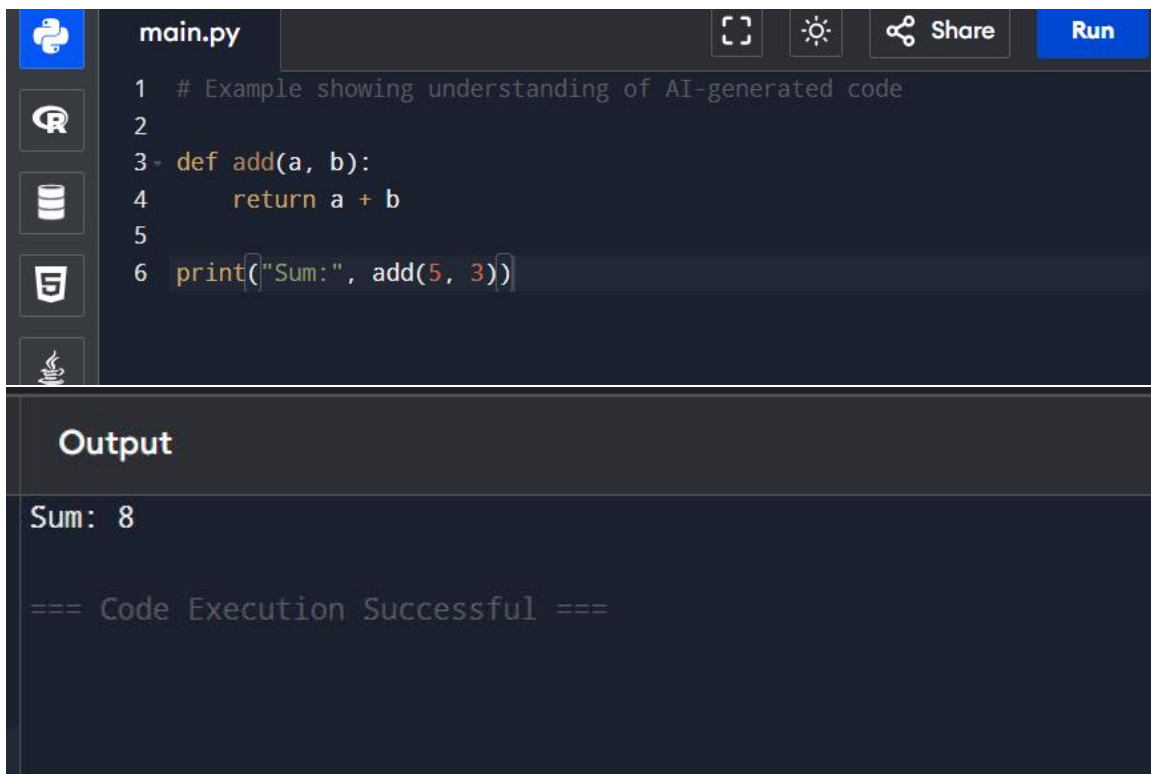
Code:

Example showing understanding of AI-generated code

```
def add(a, b):
```

```
    return a + b
```

```
print("Sum:", add(5, 3))
```



The screenshot shows a code editor interface with a dark theme. The top bar includes icons for Python, a file explorer, a search icon, a share icon, and a 'Run' button. The code editor displays the following Python code:

```
main.py
1 # Example showing understanding of AI-generated code
2
3 def add(a, b):
4     return a + b
5
6 print("Sum:", add(5, 3))
```

Below the code editor is an 'Output' section. It displays the result of the code execution:

```
Sum: 8

=== Code Execution Successful ===
```